

Slovenská technická univerzita v Bratislave
Fakulta elektrotechniky a informatiky

Pokročilé informačné technológie

Zadanie 2

Regulácia

Obsah

1 Úvod	1
1.1 Členovia tímu a rozdelenie úloh	1
2 Hardvérová časť	1
2.1 Zoznam komponentov	1
2.2 Schéma zapojenia	3
2.3 Popis TEC1-12706	3
2.4 Popis DS18B20	4
2.5 Výkonová elektronika (PWM, MOSFET)	4
2.6 Hydraulický okruh	4
3 Architektúra systému	5
3.1 UML diagram	5
3.2 Štruktúra systému	6
3.3 Sekvenčný diagram	6
3.4 Použíte protokoly	6
4 Identifikácia systému	7
4.1 Postup merania skokovej odozvy	7
4.2 Regulácia Peltierovho článku	8
4.2.1 Výpočet parametrov K a T	8
4.3 Regulácia čerpadla	8
4.3.1 Výpočet statického zosilnenia K	9
4.3.2 Výpočet časovej konštanty T a dopravného oneskorenia T_d	10
4.3.3 Výsledný model a verifikácia	11
5 Návrh regulátora	11
5.1 Mód 2 - návrh PID pre TEC	12
5.2 Mód 3 - návrh PID pre pumpu	12
5.2.1 Simulačná schéma a overenie	12
5.3 Výsledné parametre	14
6 ESP32 firmware	14
6.1 Popis kódu	14
6.2 PWM konfigurácia	15
6.3 Implementácia PID	15
6.4 MQTT komunikácia	15
6.5 Bezpečnostné prvky (saturácia, anti-windup)	15

7 Backend - Raspberry Pi	15
7.1 Flask + SocketIO	15
7.2 MQTT broker (Mosquitto)	16
7.3 SQL databáza	17
7.4 CSV + JSON archivácia	17
7.5 Thingsboard integrácia	17
8 Frontend - Webový dashboard	18
8.1 Popis funkcií (Open/Start/Stop/Close)	18
8.2 Grafy	19
8.3 Ciferníky	20
8.4 Tabuľka meraní	21
8.5 PID Panel	21
8.6 Dashboard	21
9 Výsledky merania	22
9.1 Graf regulácie Mód 2	22
9.2 Graf regulácie Mód 3	22
9.3 Zhodnotenie kvality regulácie	22
9.4 Porovnanie módov	22
10 Používateľská príručka	22
10.1 Spustenie systému (krok za krokom)	22
10.2 Popis tlačidiel dashboardu	22
10.3 Nastavenie PID parametrov	22
10.4 Export dát	22
11 Vývojárska príručka	22
11.1 Štruktúra repozitára (GitHub link)	22
11.2 Inštalácia závislostí	22
11.3 Konfigurácia (WiFi, MQTT, port)	22
11.4 Spustenie backendu	22
12 Záver	22
13 Zoznam literatúry	22

1 Úvod

Cieľom projektu je navrhnuť, zostaviť a oživiť laboratórny model na reguláciu teploty chladiacej kvapaliny v uzavretom hydraulickom okruhu pomocou Peltierovho článku. Systém je integrovaný do IoT infraštruktúry s možnosťou vzdialeného monitoringu a riadenia prostredníctvom webového dashboardu. Projekt zahŕňa identifikáciu systému, návrh PID regulátora pre dva regulačné módy a implementáciu kompletného IoT riešenia – od firmvéru mikrokontroléra až po cloudovú vizualizáciu.

1.1 Členovia tímu a rozdelenie úloh

Člen	Zodpovednosť
Ladislav Nagy	Hardvér, schéma zapojenia, oživenie komponentov
Ján Ulej	Hardvér, PWM, komunikácia
Lea Lenka Ondrejková	Identifikácia systému Mód 2, Flask backend
Kristián Pauliny	Identifikácia systému Mód 3, ThingsBoard, termoizolácia
Jozef Csabi	Backend, REST API, UML diagramy
Marián Sušina	Backend, Frontend, Flask, dokumentácia

Tabuľka 1: Rozdelenie úloh v tíme

2 Hardvérová časť

2.1 Zoznam komponentov

Systém pozostáva z dvoch kategórií komponentov. Elektronické komponenty tvoria jadro riadiaceho systému – mikrokontrolér ESP32, výkonová elektronika pre Peltierov článok a pumpu, teplotný senzor a napájanie. Hydraulické komponenty zabezpečujú cirkuláciu chladiacej kvapaliny v uzavretom okruhu. Celkové náklady na elektroniku predstavovali 62,98 €.

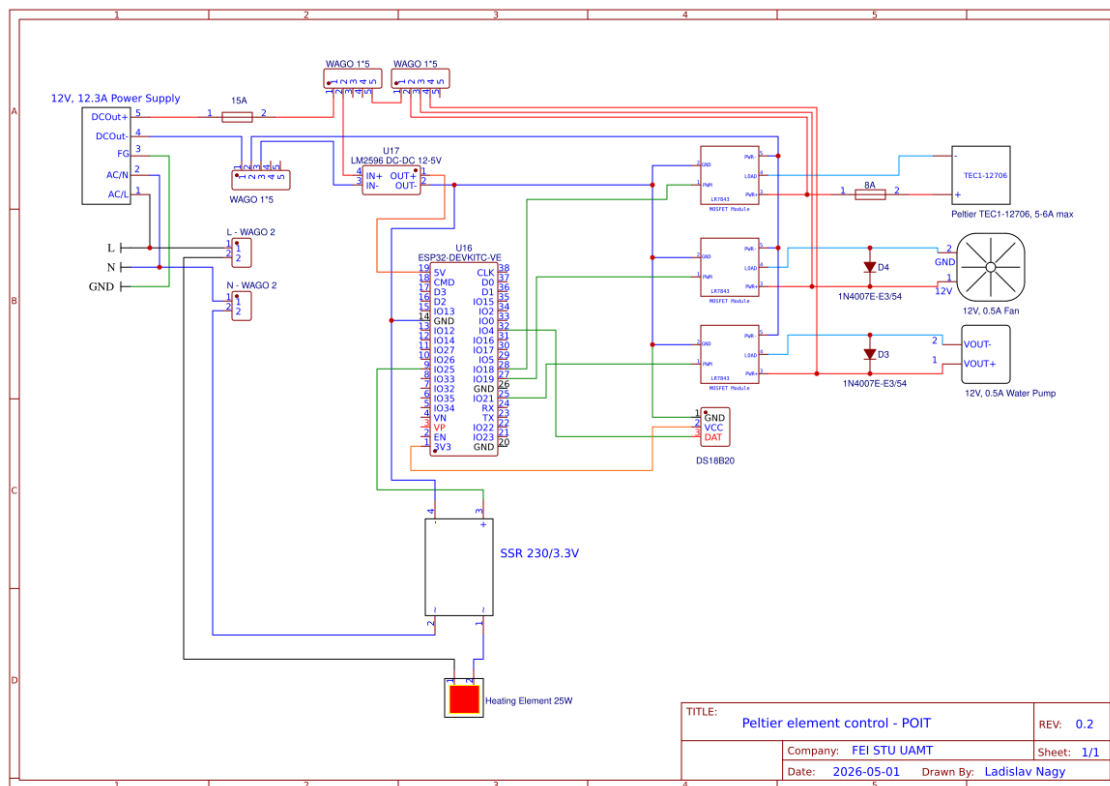
Komponent	Množstvo	Cena
ESP32-DevKitC 38pin	1	9,87 €
Zdroj 12 V / 12,5 A / 150 W	1	18,65 €
Senzor teploty DS18B20 s adaptérom	1	5,17 €
Spínač MOSFET PWM LR7843	3	3,12 €
Ohrievač Jingye 25 W (simulácia poruchy)	1	3,86 €
Solid State Relé SSR-10DA	1	6,04 €
Menič 12 V na 5 V LM2596 DC-DC	1	1,35 €
Puzdro na poistku 5×20 mm	3	2,49 €
Nepájivé kontaktné pole ZY-103	1	5,17 €
Trubičkové poistky 5×20 mm (sada)	2	1,66 €
AC napájací kábel	1	3,90 €
WAGO svorka 1×3	2	0,50 €
WAGO svorka 1×5	4	1,20 €
Spolu (elektronika)		62,98 €

Tabuľka 2: Zoznam nakúpených elektronických komponentov

Komponent	Množstvo
Destilovaná voda	3 l
Nádoba s vekom	1 ks
Hadičky 7 mm	2 × 0,5 m
Vodič 1 mm	1,5 m

Tabuľka 3: Zoznam komponentov zakúpených v OBI

2.2 Schéma zapojenia



Obr. 1: Schéma zapojenia hardvérových komponentov systému

Schéma znázorňuje prepojenie všetkých komponentov systému. ESP32 riadi výkonové prvky prostredníctvom PWM signálov cez MOSFET tranzistory. Teplotný senzor DS18B20 je pripojený na pin 4 cez OneWire zbernicu s pull-up rezistorom 4,7 kΩ. SSR relé spína ohrievač simulujúci tepelnú záťaž.

2.3 Popis TEC1-12706

Peltierov článok TEC1-12706 je termoelektrický modul využívajúci Peltierov jav na prenos tepla. Pri prechode elektrického prúdu sa jedna strana ochladzuje a druhá zahrieva. Studená strana je priložená k výmenníku tepla hydraulického okruhu, teplá strana je chladená ventilátorom s chladičom.

Parameter	Hodnota
Operačné napätie	0 – 15,2 V
Maximálny prúd	6 A
Maximálny výkon	80 W
Odporúčaný výkon	55 W
Teplotný rozsah	–30 až 70 °C

Tabuľka 4: Parametre TEC1-12706

Pri prevádzke je nevyhnutné zabezpečiť aktívne chladenie teplej strany – bez ventilátora dôjde k okamžitému tepelnému poškodeniu článku.

2.4 Popis DS18B20

DS18B20 je digitálny teplotný senzor komunikujúci cez jednodrátovú zbernicu OneWire. Senzor je ponorený priamo do chladiacej kvapaliny v expanznej nádržke, čím meria teplotu kvapaliny v uzavretom hydraulickom okruhu. Rozlíšenie 0,0625 °C pri 12-bitovom móde je dostatočné pre potreby regulácie. Čas konverzie pri maximálnom rozlíšení je 750 ms.

2.5 Výkonová elektronika (PWM, MOSFET)

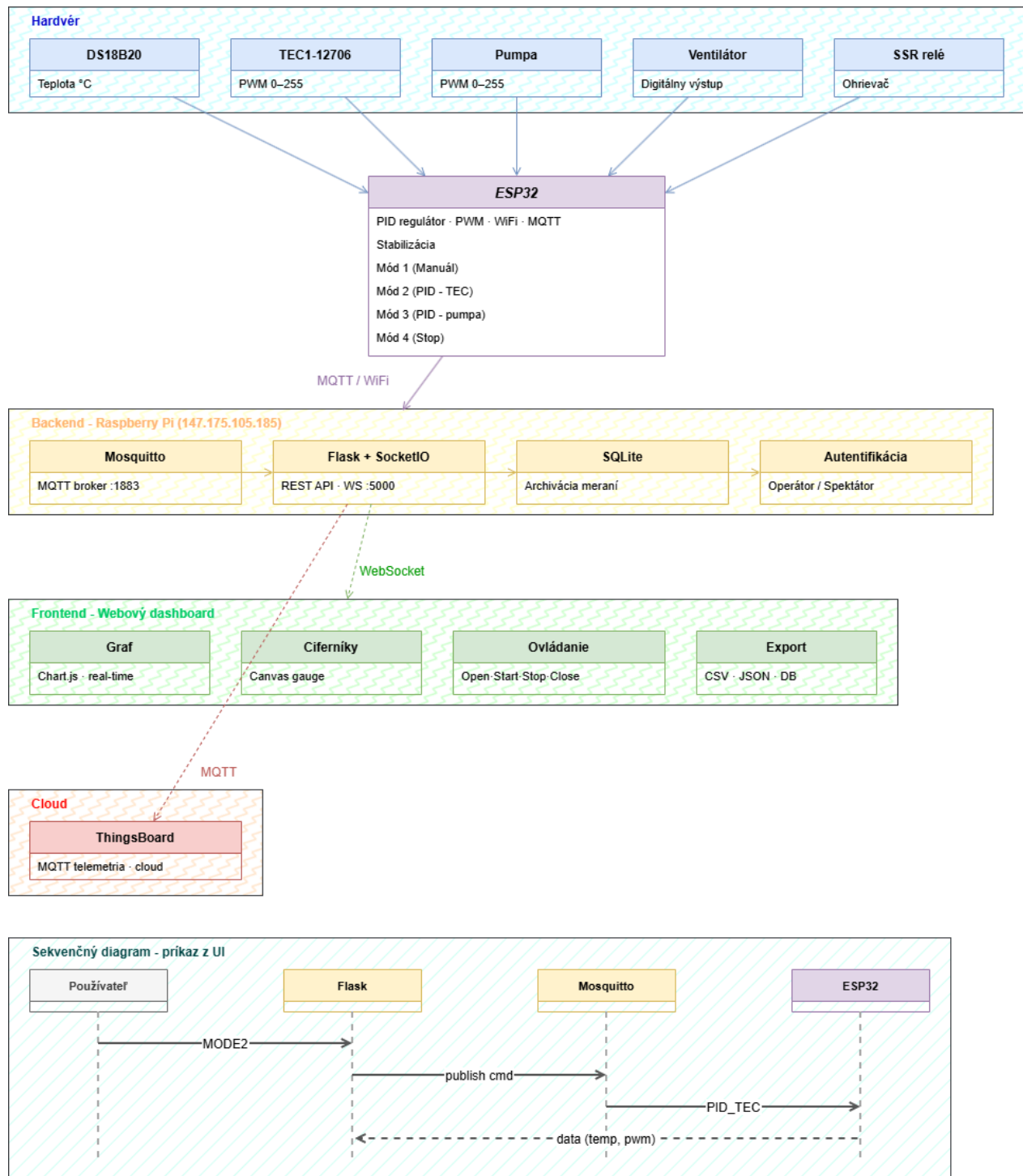
Riadenie výkonu Peltierovho článku a vodnej pumpy je realizované pomocou PWM signálov z ESP32 (frekvencia 5 kHz, 8-bitové rozlíšenie, hodnoty 0–255). Keďže ESP32 nedokáže priamo napájať výkonové prvky, sú použité MOSFET tranzistory ako spínacie prvky. PWM kanál 0 riadi TEC, PWM kanál 1 riadi pumpu. Ventilátor a SSR relé sú spínané digitálnymi výstupmi.

2.6 Hydraulický okruh

Hydraulický okruh tvorí uzavretú slučku pozostávajúcu z expanznej nádržky (0,2 l destilovanej vody), vodnej pumpy, hadičiek a výmenníka tepla prilepeného na studenú stranu Peltierovho článku. Pumpa zabezpečuje cirkuláciu kvapaliny, čím sa teplo odvádza od chladiaceho miesta k výmenníku. Celý zariadenie je tepelne izolovaný termoizolačnými taškami a bublinkovú fóliou pre minimalizáciu tepelných strát.

3 Architektúra systému

3.1 UML diagram



Obr. 2: UML diagram architektúry systému – päťvrstvová IoT architektúra so sekvenčným diagramom

3.2 Štruktúra systému

Diagram zobrazuje päť vrstiev architektúry, farebne odlíšených:

Hardvér — fyzické komponenty pripojené k ESP32: teplotný senzor DS18B20 (OneWire), Peltierov článok TEC1-12706 riadený PWM, vodná pumpa (PWM), ventilátor a SSR relé pre ohrievač.

Mikrokontrolér ESP32 — centrálny prvok implementujúci stavový automat s módmi: Mód 1 (manuál), Mód 2 (PID-TEC), Mód 3 (PID-pumpa), Stabilizácia a Stop. Vzorkuje teplotu každé 2 sekundy a komunikuje cez WiFi a MQTT.

Backend — Raspberry Pi (147.175.105.185) — skladá sa zo štyroch komponentov: Mosquitto (MQTT broker :1883), Flask + SocketIO (REST API + WebSocket :5000), SQLite (archivácia meraní) a správa rolí Operátor / Spektátor.

Frontend — Webový dashboard — dostupný na porte 5000, obsahuje Graf (Chart.js, posledných 300 bodov), Ciferníky (Canvas gauge), Ovládanie (Open / Start / Stop / Close, výber módu, nastavenie PID) a Export (CSV / JSON).

Cloud — ThingsBoard — prijíma MQTT telemetriu z backendu pre pokročilú cloudovú vizualizáciu.

3.3 Sekvenčný diagram

Sekvenčný diagram zobrazuje tok príkazu z používateľského rozhrania. Používateľ odošle príkaz `MODE2` do Flasku. Flask následne publikuje príkaz `PID_TEC` cez broker Mosquitto na ESP32. ESP32 prepne do Módu 2, spustí PID reguláciu a každé 2 sekundy odosiela späť namerané hodnoty `data` (`temp`, `pwm`).

3.4 Použité protokoly

Protokol	Port	Použitie
MQTT	1883	ESP32 ↔ Mosquitto ↔ Flask ↔ ThingsBoard — telemetria a príkazy
WebSocket	5000	Flask → Dashboard — real-time push dát bez pollingu
REST API	5000	Dashboard → Flask — ovládanie, nastavenie PID, export
OneWire	—	ESP32 ← DS18B20 — meranie teploty
PWM	—	ESP32 → TEC / Pumpa — riadenie výkonu (0–255)

Tabuľka 5: Prehľad komunikačných protokolov

MQTT je asynchrónny publish/subscribe protokol s nízkou réžiou, ideálny pre IoT zariadenia s obmedzeným výkonom. WebSocket udržiava trvalé spojenie medzi serverom a prehliadačom pre okamžitú aktualizáciu dashboardu. REST API zabezpečuje synchrónne ovládanie systému z frontendu.

4 Identifikácia systému

V tejto časti bude opísaný spôsob identifikácie systému pre dva módy regulácie. Pred samotným opisom metodiky riešenia je potrebné poznamenať, že celý systém bol mierne upravený oproti stavu, v akom bol zostavený našimi kolegami. Boli realizované tri hlavné úpravy.

Prvou úpravou bola zmena objemu destilovanej vody, ktorú sme chladili. Pôvodne sme mali v úmysle chladiť približne 2 litre vody. Ako zásobník vody slúžil plastový kýbeľ. Ten sme postupne nahrádzali zaváraninovými fľašami rôznych objemov. Nakoniec sme použili malú zaváraninovú fľašu s objemom 0,2 litra.

Druhou úpravou bola tepelná izolácia zásobníka destilovanej vody. Na tento účel sme použili niekoľko termoizolačných tašiek. Zásobník vody spolu s čerpadlom a výhrevným telesom (poruchou) bol vložený do troch termoizolačných tašiek.

Posledným vylepšením bolo obalenie hadičiek na vodu a samotnej jednotky Peltierovho článku malými uterákmi a bublinkovou fóliou. Aj tieto úpravy boli realizované s cieľom zlepšiť tepelnú izoláciu systému.

Ukázalo sa, že všetky tieto úpravy pozitívne prispeli k rýchlejšej stabilizácii výstupnej veličiny – teploty vody v zásobníku.

4.1 Postup merania skokovej odozvy

Meranie teploty destilovanej vody v zásobníku na účely identifikácie sme realizovali podľa nasledujúceho postupu. Najprv sme do zásobníka naliali 0,2 litra vody. Zásobník sme uzavreli a vložili do troch termoizolačných tašiek. Napájací zdroj celého systému sme pripojili do elektrickej siete. Mikrokontrolér NodeMCU sme následne pripojili k osobnému počítaču cez USB.

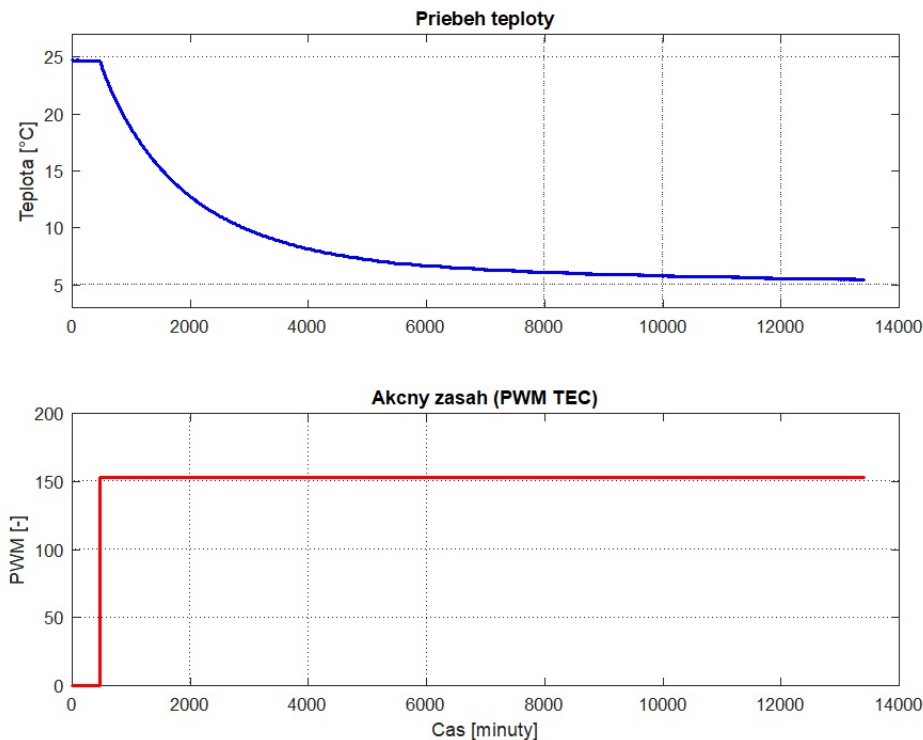
Do mikrokontroléra sme nahrali špeciálny identifikačný skript, ktorého úlohou bolo merať teplotu vody v zásobníku a podľa zvoleného režimu ovládať akčné členy systému, konkrétne Peltierov článok, čerpadlo a ventilátor chladiča na teplej strane Peltierovho článku. Relevantné údaje boli každé dve sekundy odosielané po sériovej linke. Tieto údaje sme priebežne zaznamenávali, vizualizovali v grafoch pomocou skriptu v jazyku Python a zároveň ukladali do súboru typu `.csv`.

Prostredníctvom sériovej linky bolo možné odoslať príkazy pre tri rôzne režimy merania: *stabilizácia*, *skok* a *zastaviť všetko*. Stabilizácia predstavovala prvú fázu merania, počas ktorej bola akčná veličina nastavená na nulovú hodnotu a čakalo sa na ustálenie výstupnej veličiny. Počas skoku bola akčná veličina nastavená na svoju maximálnu hodnotu a následne sa čakalo, kým sa výstupná veličina ustáli na novej hodnote. Príkaz *zastaviť všetko* ukončil celé meranie.

4.2 Regulácia Peltierovho článku

Identifikačné meranie pri regulácii teploty vody pomocou Peltierovho článku bolo realizované nasledovne. Chladič na teplej strane Peltierovho článku bol počas celého merania zapnutý na maximálny výkon. Vodné čerpadlo bolo taktiež zapnuté konštantne s PWM plnením 50%. Počas fázy stabilizácie bol Peltierov článok vypnutý (PWM plnenie 0%). Po ustálení teploty na stabilnej hodnote bol realizovaný skok akčnej veličiny, pričom PWM plnenie Peltierovho článku bolo nastavené na 60%.

Priebeh odozvy systému na skokovú zmenu výkonu Peltierovho článku je znázornený na obrázku 3.



Obr. 3: Priebeh odozvy teploty na skokovú zmenu výkonu Peltierovho článku

4.2.1 Výpočet parametrov K a T

Lea

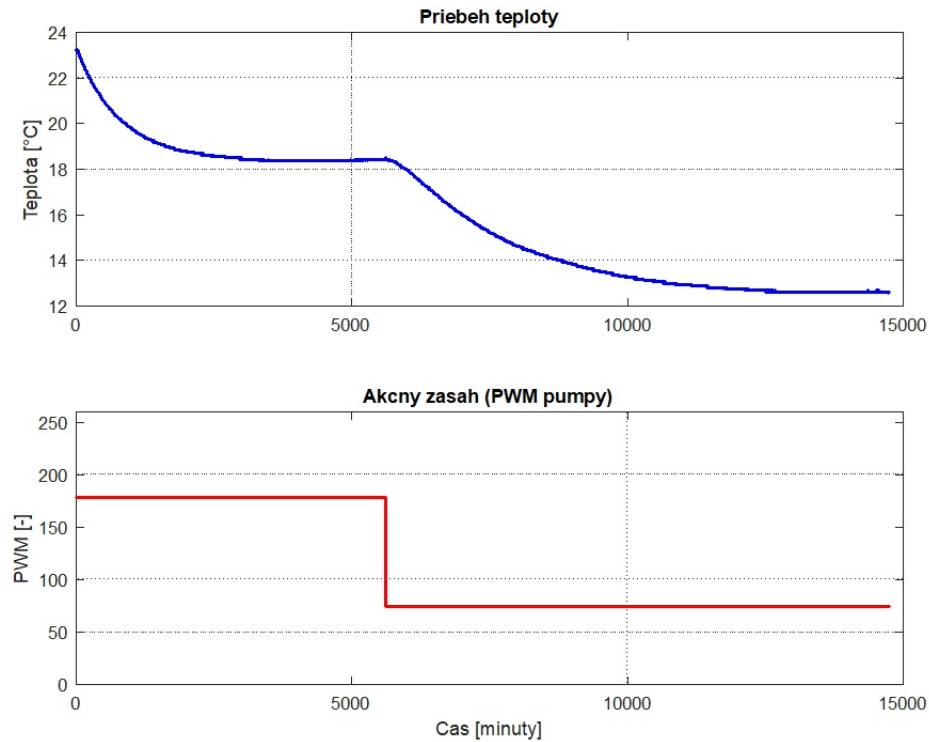
4.3 Regulácia čerpadla

Pri regulácii teploty vody pomocou vodného čerpadla bol chladič na teplej strane Peltierovho článku zapnutý konštantne na maximálny výkon. Peltierov článok bol počas celého merania zapnutý s PWM plnením 25 %.

Počas stabilizačnej fázy bolo čerpadlo nastavené na PWM plnenie 30%. Vo fáze skoku bolo PWM plnenie zvýšené na 70%. Skok akčnej veličiny nebol realizovaný z hodnoty 0%, pretože pri takomto

nastavení by sa čerpadlo zastavilo a ustálená hodnota získaná počas stabilizačnej fázy by nebola reprezentatívna. PWM plnenie 30% predstavovalo najnižšiu hodnotu, pri ktorej bolo čerpadlo schopné spoľahlivo sa rozbehnúť a zostať v prevádzke počas celého merania.

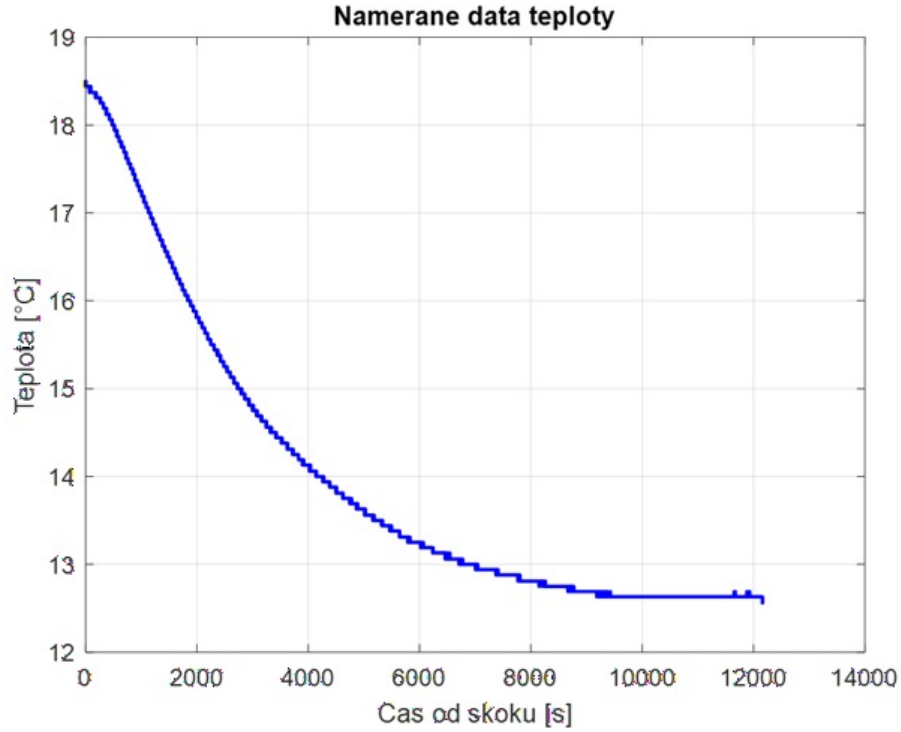
Priebeh odozvy teploty vody v zásobníku na skokovú zmenu výkonu čerpadla je znázornený na obrázku 4.



Obr. 4: Priebeh odozvy teploty na skokovú zmenu výkonu čerpadla

4.3.1 Výpočet statického zosilnenia K

Skoková zmena akčného zásahu bola realizovaná znížením plnenia PWM čerpadla z pôvodnej hodnoty $u_0 = 179$ na $u_\infty = 75$. Zníženie prietoku spôsobilo výraznejšie ochladzovanie vody, pričom teplota klesla z ustálenej počiatočnej hodnoty $y_0 = 18.44^\circ\text{C}$ na novú ustálenú hodnotu $y_\infty = 12.63^\circ\text{C}$. Priebeh tejto skokovej odozvy orezaný na relevantnú časť pre identifikáciu je znázornený na obrázku 5.



Obr. 5: Priebeh odozvy systému použitý na identifikáciu

Statické zosilnenie K sme vypočítali priamo z týchto fyzikálnych hodnôt:

$$K = \frac{\Delta y}{\Delta u} = \frac{y_{\infty} - y_0}{u_{\infty} - u_0} = \frac{12.63 - 18.44}{75 - 179} = \frac{-5.81}{-104} \approx 0.0559^{\circ}\text{C}/\text{PWM} \quad (1)$$

4.3.2 Výpočet časovej konštanty T a dopravného oneskorenia T_d

Pre určenie dynamických parametrov bola využitá **dvojbodová metóda**. Keďže reálna teplota počas merania klesala, namerané dáta boli najskôr invertované a normalizované do rozsahu $\langle 0, 1 \rangle$. Následne boli z normalizovanej odozvy odčítané časy, v ktorých výstup dosiahol 28.3 % a 63.2 % svojej ustálenej hodnoty:

- $t_1 = 1293.2 \text{ s}$ (pre 28.3 %)
- $t_2 = 2996.8 \text{ s}$ (pre 63.2 %)

Na základe fyzikálneho pozorovania systému bolo dopravné oneskorenie manuálne stanovené na $T_d = 72 \text{ s}$. Časová konštanta T bola následne dopočítaná tak, aby model presne prechádzal bodom t_2 :

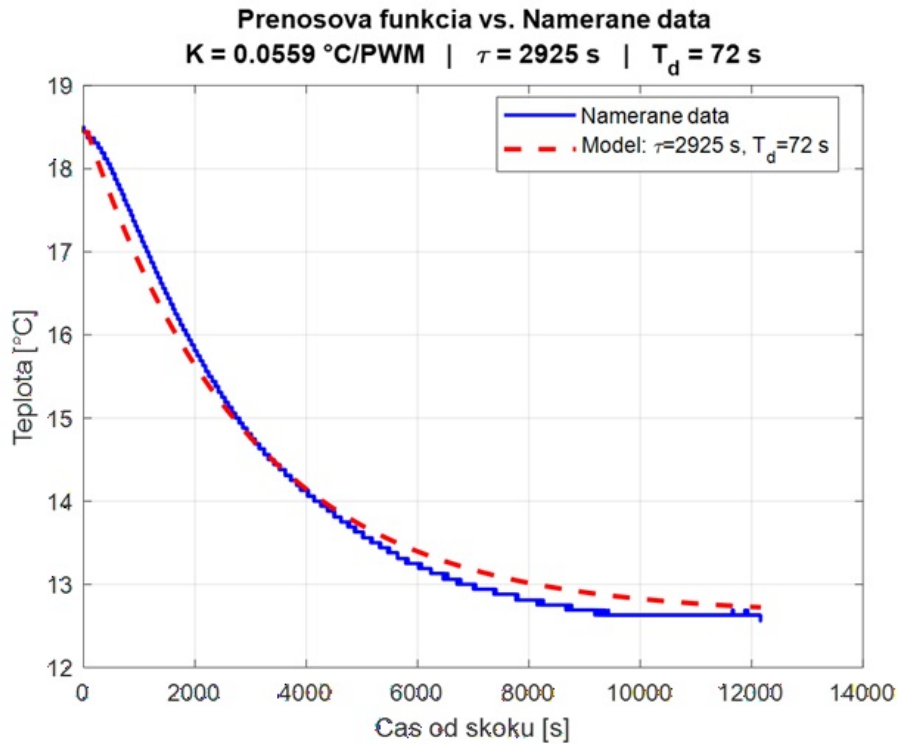
$$T = t_2 - T_d = 2996.8 - 72 = 2924.8 \text{ s} \quad (2)$$

4.3.3 Výsledný model a verifikácia

Po dosadení vypočítaných parametrov má výsledná prenosová funkcia identifikovaného systému tvar:

$$G(s) = \frac{0.0559}{2924.8s + 1} e^{-72.0s} \quad (3)$$

Správnosť identifikovanej prenosovej funkcie bola graficky overená v prostredí Simulink. Na obrázku 6 je zobrazené priame porovnanie nameraných dát z reálneho systému s teoretickou skokovou odozvou navrhnutého modelu $G(s)$. Z grafu je zrejmé, že prenosová funkcia verne zachytáva dynamiku aj statické zosilnenie reálneho systému.



Obr. 6: Porovnanie nameranej odozvy reálneho systému s identifikovaným modelom

5 Návrh regulátora

Na základe matematických modelov vo forme prenosovej funkcie $G(s)$, ktoré boli odvodené v predchádzajúcej fáze identifikácie, sme pristúpili k samotnému návrhu regulátorov. Využitie tohto exaktného analytického popisu systému nám umožnilo namiesto experimentálneho ladenia metódou pokus-omyl exaktne vypočítať optimálne parametre regulátora. Cieľom návrhu bolo zabezpečiť stabilnú odozvu uzavretého regulačného obvodu, minimalizovať prechod a dosiahnuť presné sledovanie žiadanej teploty reálneho systému.

5.1 Mód 2 - návrh PID pre TEC

5.2 Mód 3 - návrh PID pre pumpu

Pre riadenie procesu chladenia bola zvolená štruktúra PI regulátora. Derivačnú zložku sme vynechali, keďže ide o pomalý tepelný systém a zosilňovanie šumu z merania by spôsobovalo nežiaduce kmitanie akčného zásahu.

Parametre regulátora boli analyticky vypočítané metódou SIMC (Skogestad). Pre zaistenie stability regulačného pochodu bol zvolený ladiaci parameter $\tau_c = 6T_d = 432$ s. Po dosadení parametrov identifikovaného modelu ($K = 0.0559$, $T = 2924.8$ s, $T_d = 72$ s) sme získali:

$$K_p = \frac{T}{K(\tau_c + T_d)} = \frac{2924.8}{0.0559 \cdot (432 + 72)} \approx 103.81 \quad (4)$$

$$T_i = \min(T, 4(\tau_c + T_d)) = 2016 \text{ s} \quad (5)$$

Pre paralelný tvar regulátora sme dopočítali integračnú konštantu:

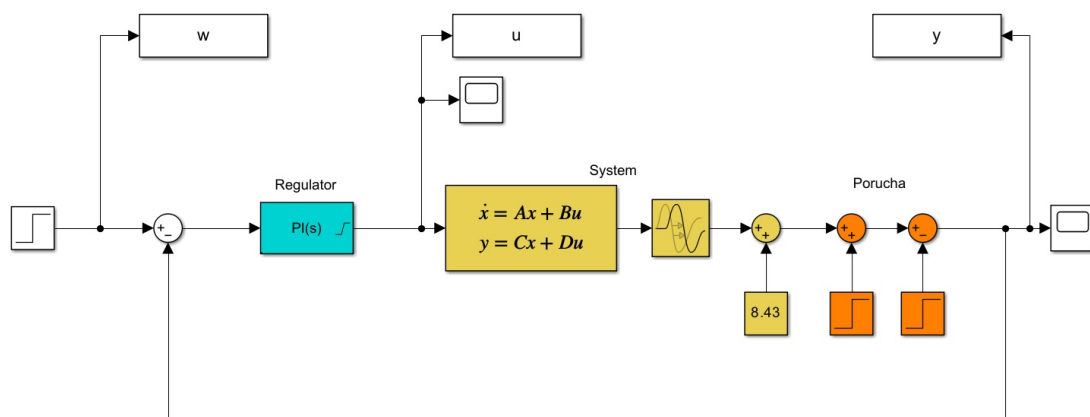
$$K_i = \frac{K_p}{T_i} = \frac{103.81}{2016} \approx 0.0515 \quad (6)$$

5.2.1 Simulačné schéma a overenie

Funkčnosť regulátora bola overená v prostredí Simulink (Obrázok 7). Model vo forme prenosovej funkcie bol nahradený blokom stavového priestoru (*State-Space*), čo umožnilo inicializovať systém na reálnu počiatočnú teplotu (24°C).

K výstupu dynamického modelu bol pripočítaný teplotný offset (Bias) 8.43°C . Táto hodnota predstavuje ustálenú teplotu pri nulovom výkone čerpadla a zabezpečuje, že regulátor pracuje v absolútnych fyzikálnych hodnotách, vďaka čomu správne reaguje na saturačné limity PWM signálu.

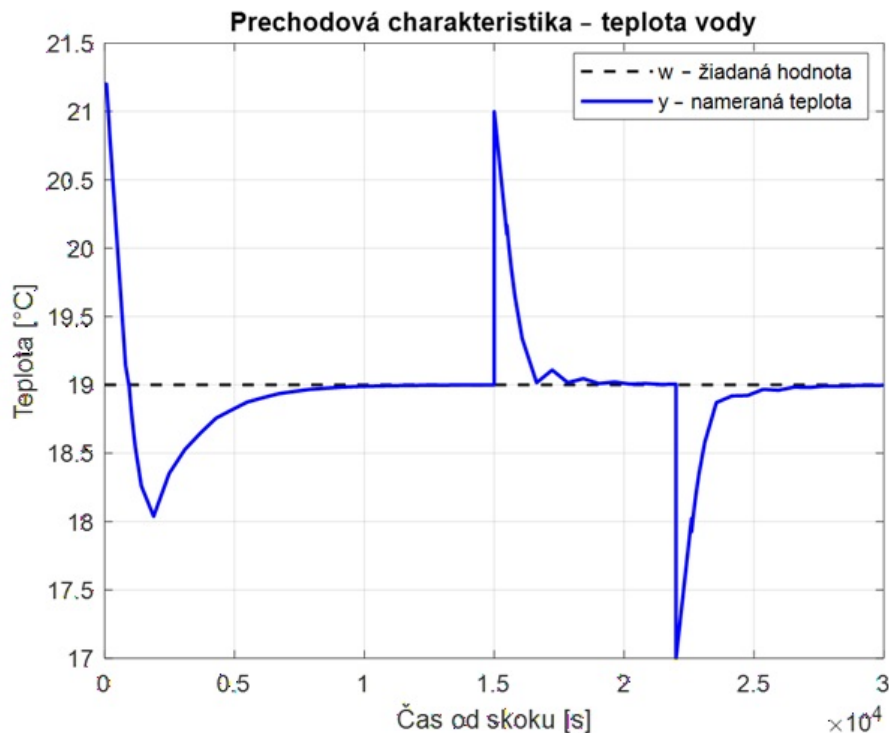
Na overenie robustnosti riadenia bola do schémy priamo na výstup systému pridaná skoková porucha. Simulácia potvrdila, že navrhnutý PI regulátor dokáže túto poruchu úspešne kompenzovať a systém spoľahlivo vrátiť na žiadanú hodnotu teploty.



Obr. 7: Simulačná schéma uzavretého regulačného obvodu s modelovaním poruchy

Cieľom simulácie bolo overiť schopnosť regulátorov dosiahnuť a udržať novú žiadanú hodnotu teploty pri rešpektovaní reálnych saturačných obmedzení akčného člena.

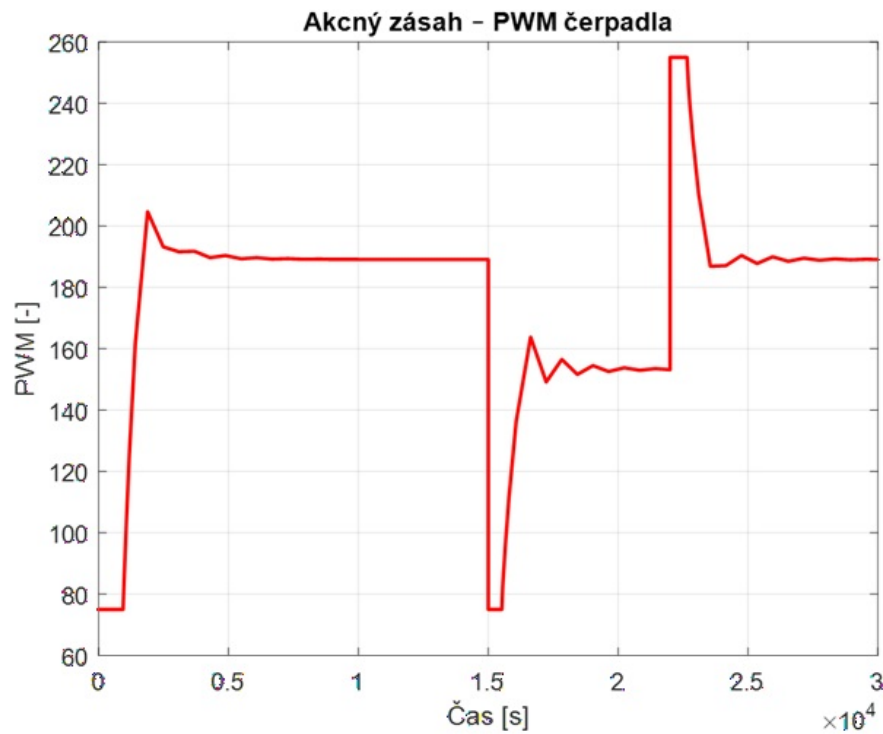
Po spustení simulácie boli zaznamenané časové priebehy regulovanej veličiny a akčného zásahu. Na obrázku 8 je zobrazená prechodová charakteristika uzavretého regulačného obvodu. Z grafu je zrejmé, že teplota sleduje referenčný signál a po prechodnom deji s úvodným prekmitom sa stabilne ustáľuje na žiadanej hodnote.



Obr. 8: Prechodová charakteristika uzavretého regulačného obvodu (regulovaná teplota)

Na obrázku 9 je následne znázornený priebeh akčného zásahu v čase. Vďaka aplikovanému *anti-*

windup mechanizmu a správnej inicializácii pracovného bodu zostáva akčný zásah po celý čas v povolených fyzikálnych medziach. Zásah plynule reaguje na regulačnú odchýlku a nevykazuje vysokofrekvenčné kmitanie, čo je kľúčové pre dlhodobú životnosť reálneho čerpadla.



Obr. 9: Priebeh akčného zásahu regulátora (PWM signál čerpadla)

Na základe dosiahnutých výsledkov možno konštatovať, že navrhnutý PI regulátor je stabilný, dostatočne robustný a je plne schopný úspešne regulovať identifikovaný simulačný model procesu chladenia.

5.3 Výsledné parametre

6 ESP32 firmware

6.1 Popis kódu

Firmvér je implementovaný v jazyku C++ pomocou frameworku Arduino pre ESP32 v prostredí PlatformIO. Jadrom firmvéru je stavový automat s piatimi stavmi: `MODE_STOP`, `MODE_STABILIZE`, `MODE_PID_TEC`, `MODE_PID_PUMP` a `MODE_MANUAL`. Hlavná slučka beží s periódou 5 sekundy, počas ktorej sa vykoná meranie teploty, výpočet regulátora a publikovanie dát cez MQTT.

6.2 PWM konfigurácia

ESP32 využíva vlastný LEDC systém pre generovanie PWM signálov. Sú nakonfigurované dva kanály – kanál 0 pre Peltierov článok (pin 18) a kanál 1 pre pumpu (pin 21). Oba kanály pracujú s frekvenciou 5 kHz a 8-bitovým rozlíšením (hodnoty 0–255), čo zodpovedá rozsahu výkonu 0–100 %.

```
ledcSetup(CH_PELTIER, 5000, 8);  
ledcAttachPin(PIN_PELTIER, CH_PELTIER);
```

6.3 Implementácia PID

Systém implementuje dva nezávislé PID regulátory. Pre Múd 2 (regulácia TEC) sú parametre $K_p = -50$, $K_i = -0,053$, $K_d = -25$. Pre Múd 3 (regulácia pumpy) je použitý PI regulátor s Feed-Forward a Anti-Windup s parametrami $K_p = 103,81$, $K_i = 0,0515$.

PID výpočet prebieha v diskretnom čase s periódou vzorkovania $T_s = 5$ s. Derivačná zložka využíva diferenciu po chybe medzi dvoma po sebe idúcimi vzorkami.

6.4 MQTT komunikácia

Po pripojení na WiFi sieť sa ESP32 pripojí na MQTT broker Mosquitto bežiaci na Raspberry Pi. ESP32 publikuje namerané dáta na tému `peltier/data` vo formáte CSV a odoberá príkazy z témy `peltier/cmd`. Pri výpadku spojenia sa ESP32 automaticky pokúša o opätovné pripojenie.

6.5 Bezpečnostné prvky (saturácia, anti-windup)

Akčný zásah je obmedzený saturáciou na maximálne 80 % výkonu ($PWM = 204$) pre Múd 2, čím sa predchádza prehriatiu Peltierovho článku. Pre Múd 3 sú definované limity `min_pwm = 75` a `max_pwm = 180`. Anti-windup je riešený metódou clampingu – integrál sa akumuluje len v prípade, že akčný zásah nie je v saturácii, prípadne ak smer chyby prispieva k opusteniu saturácie. Pri štarte každého módu sa integrál a predchádzajúca chyba vynulujú.

7 Backend - Raspberry Pi

Backend systému beží na jednodoskovom počítači Raspberry Pi 4B (verejná IP adresa 147.175.105.185) a tvorí centrálnu komunikačnú a dátovú vrstvu celého IoT riešenia. Zodpovedá za príjem dát z ESP32, ich archiváciu, distribúciu do frontendu v reálnom čase a odosielanie príkazov späť na mikrokontrolér.

7.1 Flask + SocketIO

Webový server je implementovaný pomocou mikroframeworku **Flask** pomocou knižnice **Flask-SocketIO**. Flask zabezpečuje klasické HTTP endpointy, zatiaľ čo SocketIO umožňuje obojsmernú

komunikáciu v reálnom čase medzi serverom a prehliadačom bez nutnosti opakovania HTTP požiadaviek.

Server počúva na porte 5000 a je nakonfigurovaný pre viacvláknový asynchrónny režim (`async_mode='threading'`), čo umožňuje súbežné spracovanie MQTT správ a WebSocket udalostí.

Kľúčové SocketIO udalosti:

- `command` – príjem príkazov z frontendu (OPEN, START, STOP, CLOSE, MODE1/2/3, SET_PID),
- `data` – vysielanie nameraných hodnôt do všetkých pripojených klientov,
- `system_status` – informácia o stave systému (inicializovaný / beží),
- `pid_updated` / `pid_error` – potvrdenie alebo odmietnutie zmeny PID parametrov.

Autentifikácia je riešená pomocou **Flask session** s dvoma rolami: *operator* (plný prístup – ovládanie aj sledovanie) a *spectator* (iba sledovanie). Prístup k chráneným endpointom je vynucovaný dekorátorom `login_required`.

7.2 MQTT broker (Mosquitto)

Komunikácia medzi ESP32 a Raspberry Pi prebieha prostredníctvom protokolu **MQTT**. Na Raspberry Pi beží broker **Mosquitto**, na ktorý sa ESP32 pripája cez WiFi. Flask backend si udržiava vlastného MQTT klienta (`paho-mqtt`), ktorý je prihlásený na odber témy `peltier/data` a zároveň publikuje príkazy na tému `peltier/cmd`.

Formát správy z ESP32 je textový CSV riadok:

```
<time_ms>,<temp_C>,<tec_pwm>,<pump_pwm>,<mode>
```

Správy začínajúce znakom `#` a riadky s menej ako 5 poľami sú ignorované. Hodnoty teploty pod `-100 °C` sú považované za chybu senzora a tiež sa zahadzujú.

Príkazy posielať na ESP32:

- `STABILIZE` – ustálenie systému po štarte (TEC 25 %, pumpa 50 %),
- `MANUAL:tec:pump` – manuálne nastavenie PWM hodnôt (Mód 1),
- `PID_TEC` – spustenie PID regulácie TEC (Mód 2),
- `PID` – spustenie PID regulácie pumpy (Mód 3),
- `SET_SP` / `SET_KP` / `SET_KI` / `SET_KD` – aktualizácia parametrov regulácie,
- `STOP` – zastavenie všetkých výstupov.

7.3 SQL databáza

Každá prijatá meracia správa je uložená do **SQLite** databázy (**peltier.db**) prostredníctvom ORM knižnice **Flask-SQLAlchemy**. Databáza je jediným zdrojom pravdy (*single source of truth*) pre historické dáta.

Tabuľka **Measurement** obsahuje nasledujúce stĺpce:

Stĺpec	Typ	Popis
id	INTEGER	Primárny kľúč (auto-increment)
time_ms	INTEGER	Čas od štartu merania v milisekundách
temp_C	FLOAT	Teplota kvapaliny v °C
tec_pwm	INTEGER	PWM hodnota TEC (0–255)
pump_pwm	INTEGER	PWM hodnota pumpy (0–255)
mode	TEXT	Aktívny mód (MODE1 / MODE2 / MODE3 / STOP)
created	DATETIME	Čas záznamu (UTC)

Tabuľka 6: Štruktúra tabuľky **Measurement** v SQLite databáze

REST endpoint `/api/history` vracia posledných 500 záznamov zoradených chronologicky – tieto dáta sa načítavajú pri inicializácii grafu na dashboarde.

7.4 CSV + JSON archivácia

Backend umožňuje export celej databázy meraní vo formátoch **CSV** a **JSON** prostredníctvom HTTP endpointov `/api/download/csv` a `/api/download/json`. Súborý sú generované na požiadanie, pomenované s časovou pečiatkou spustenia servera (napr. `export_20260530_143000.csv`) a uložené do adresára `data`.

CSV súbor obsahuje stĺpce: `time_ms`, `temp_C`, `tec_pwm`, `pump_pwm`, `mode`, `created`. JSON súbor obsahuje rovnaké polia vo forme poľa objektov s kľúčom `ts` pre časovú pečiatku. Exportovaný súbor je vrátený ako príloha (`as_attachment=True`), čo umožňuje jeho priame stiahnutie z prehliadača.

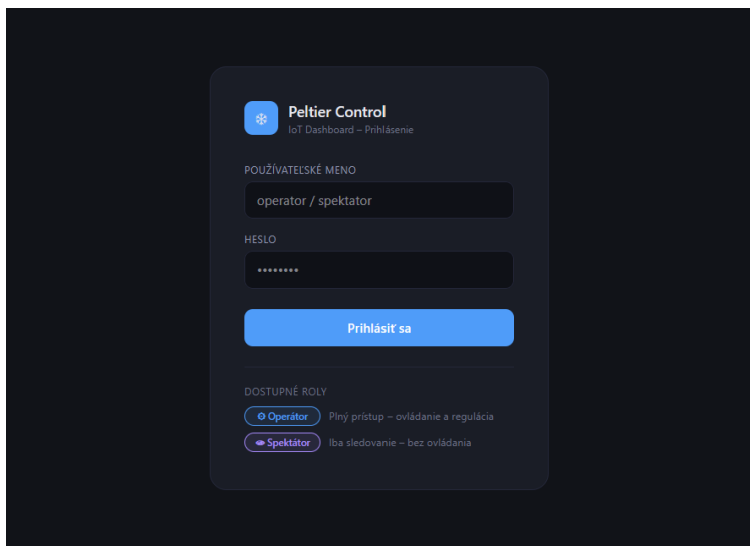
7.5 Thingsboard integrácia

Backend obsahuje implementovanú integráciu s cloudovou platformou **ThingsBoard** cez MQTT protokol. Po každom prijatom meraní sú hodnoty teploty, TEC PWM, pumpa PWM, aktívneho módu a setpointu publikované na ThingsBoard server (`mqtt.eu.thingsboard.cloud`) pomocou access tokenu zariadenia.

Platformu sa v rámci projektu nepodarilo plne sprevádzkovať do finálnej verzie. Kód integrácie je však súčasťou backendu a po správnom nastavení prístupových údajov je funkčný.

8 Frontend - Webový dashboard

Webový dashboard je prístupný na adrese `http://147.175.105.185:5000` a tvorí hlavné používateľské rozhranie systému. Je implementovaný v čistom HTML, CSS a JavaScripte bez externých SPA frameworkov, pričom na vizualizáciu využíva knižnice **Chart.js** (grafy) a **canvas-gauges** (ciferníky). Komunikácia so serverom prebieha cez WebSocket (Socket.IO klient).



Obr. 10: Prihlasovacia stránka – Peltier Control IoT Dashboard

8.1 Popis funkcií (Open/Start/Stop/Close)

Ovládanie systému je rozdelené do štyroch hlavných tlačidiel umiestnených v hornej lište dashboardu, ktoré musia byť aktivované v predpísanom poradí:

1. **Open** – inicializuje spojenie so serverom a spustí MQTT listener pre príjem dát z ESP32. Systém prejde do stavu *inicializovaný*.
2. **Start** – odošle príkaz **STABILIZE** na ESP32 a spustí zber meraní. ESP32 nastaví TEC na 25 % a pumpu na 50 % na ustálenie systému.
3. **Stop** – zastaví meranie, odošle **STOP** na ESP32 a vynuluje PWM hodnoty. Systém zostane v inicializovanom stave.
4. **Close** – ukončí spojenie, deaktivuje systém a odošle **STOP** ako bezpečnostné opatrenie.

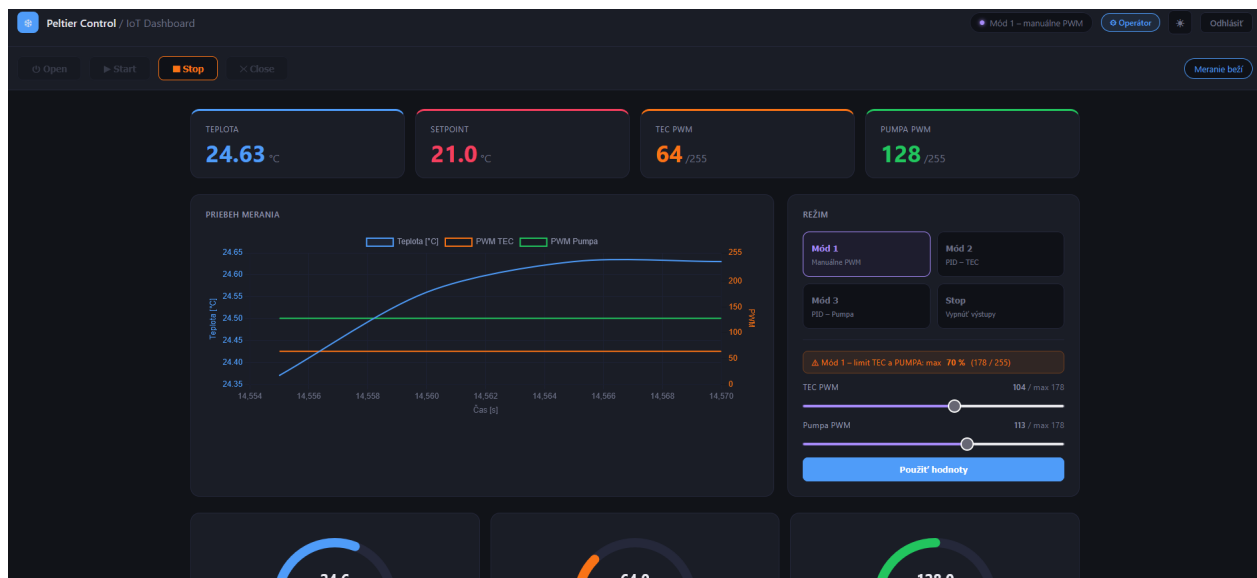
Grafické zvýraznenie aktívneho tlačidla (napr. oranžový rámček pri *Stop* počas behu merania) poskytuje operátorovi okamžitú spätnú väzbu o stave systému.

8.2 Grafy

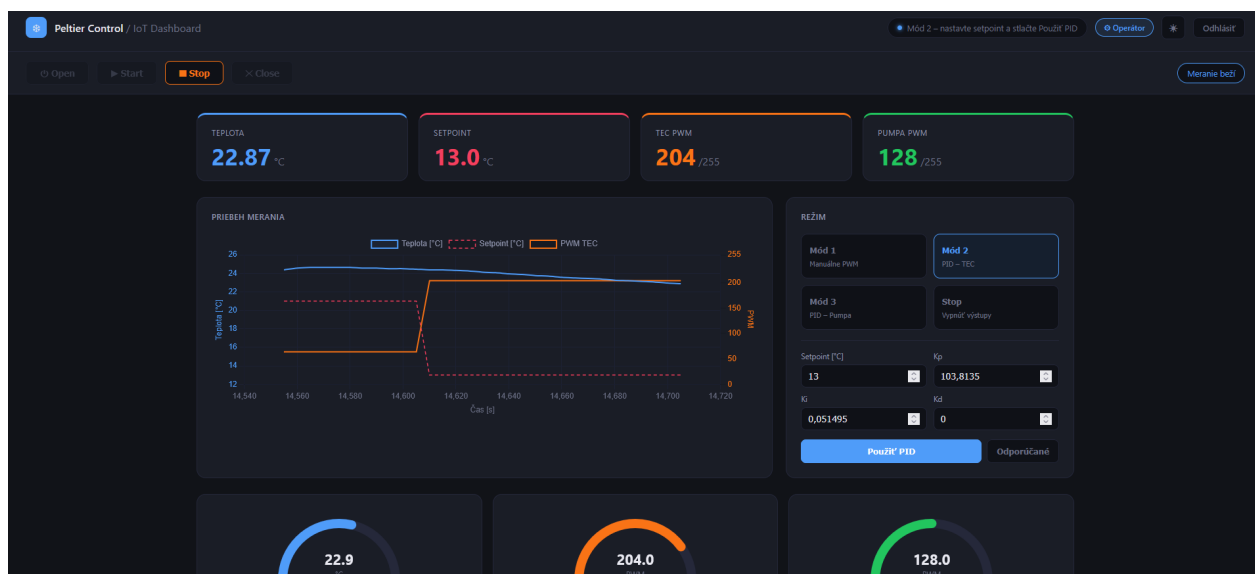
Sekcia **Priebeh merania** zobrazuje interaktívny čiarový graf vykreslený knižnicou Chart.js. Graf zobrazuje časové priebehy:

- teploty kvapaliny [°C] – modrá krivka,
- setpointu [°C] – červená prerušovaná krivka,
- PWM TEC alebo PWM Pumpy v závislosti od aktívneho módu – oranžová/zelená krivka (pravá os Y, rozsah 0–255).

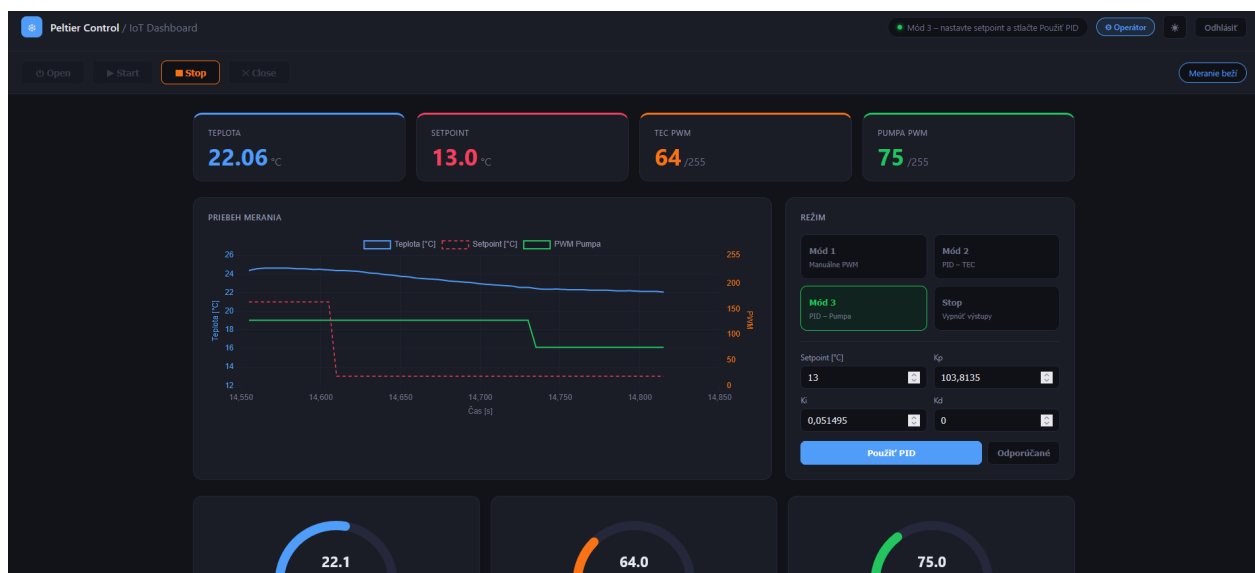
Graf si uchováva posledných 300 bodov a je aktualizovaný v reálnom čase pri každom príchode WebSocket správy. Po načítaní stránky sa zobrazia historické dáta z databázy (endpoint `/api/history`).



Obr. 11: Dashboard v Móde 1 (manuálne PWM) – priebeh merania s grafom teploty a PWM



Obr. 12: Dashboard v Móde 2 (PID – TEC) – regulácia TEC PWM s viditeľným skokom setpointu



Obr. 13: Dashboard v Móde 3 (PID – Pumpa) – regulácia prietoku pumpy

8.3 Ciferníky

Pod grafom sú umiestnené tri analógové ciferníky (gauges) implementované pomocou knižnice **canvas-gauges**:

- **Teplota [°C]** – modrý ciferník, rozsah 0–40 °C,
- **TEC PWM** – oranžový ciferník, rozsah 0–255,
- **Pumpa PWM** – zelený ciferník, rozsah 0–255.

Ciferníky sú aktualizované synchronne s grafom a poskytujú intuitívnu vizuálnu reprezentáciu aktuálnych hodnôt.



Obr. 14: Analógové ciferníky – teplota, TEC PWM a Pumpa PWM

8.4 Tabuľka meraní

Dashboard obsahuje tabuľku posledných 50 meraní načítaných z REST endpointu `/api/history`. Každý riadok zobrazuje čas záznamu (UTC), teplotu, TEC PWM, pumpa PWM a aktívny mód. Tabuľka je statická – neaktualizuje sa automaticky, ale je obnovená pri každom načítaní stránky.

8.5 PID Panel

Panel **Režim** na pravej strane dashboardu umožňuje operátorovi:

- prepínanie medzi módmi (Mód 1, Mód 2, Mód 3, Stop),
- zadanie hodnôt *Setpoint*, K_p , K_i , K_d do číselných polí,
- odoslanie parametrov tlačidlom **Použiť PID**,
- načítanie odporúčaných parametrov tlačidlom **Odporúčané** (rôzne hodnoty pre Mód 2 a Mód 3 podľa identifikácie systému).

Ak operátor zadá setpoint vyšší ako aktuálna teplota, server požiadavku odmietne a odošle chybovú správu `pid_error` – systém totiž dokáže iba chladiť. Aktívny mód je vizuálne zvýraznený farebným rámčekom príslušného tlačidla.

8.6 Dashboard

Celkový dizajn dashboardu je navrhnutý s dôrazom na prehľadnosť a použiteľnosť v laboratórnych podmienkach. Rozhranie podporuje prepínanie medzi **tmavým a svetlým režimom** (Dark/Light mode), pričom predvolený je tmavý. Používateľ je informovaný o aktuálnom stave systému prostredníctvom:

- farebných informačných kariet v hornej časti (Teplota, Setpoint, TEC PWM, Pumpa PWM) s farebnými rámčekom podľa veličiny,
- stavového indikátora v pravom rohu lišty (*Meranie beží* / neaktívne),

- zobrazeného mena a roly prihláseného používateľa,
- hlásenia aktívneho módu v hornej lište.

Rozhranie pre rolu *spectator* je identické, no všetky ovládacie prvky (tlačidlá, posúvače, PID polia) sú deaktivované.

9 Výsledky merania

9.1 Graf regulácie Mód 2

9.2 Graf regulácie Mód 3

9.3 Zhodnotenie kvality regulácie

9.4 Porovnanie módov

10 Používateľská príručka

10.1 Spustenie systému (krok za krokom)

10.2 Popis tlačidiel dashboardu

10.3 Nastavenie PID parametrov

10.4 Export dát

11 Vývojárska príručka

11.1 Štruktúra repozitára (GitHub link)

11.2 Inštalácia závislostí

11.3 Konfigurácia (WiFi, MQTT, port)

11.4 Spustenie backendu

12 Záver

13 Zoznam literatúry